

Bridge Policy Scenarios

Stand: 2026-05-10

Dieses Dokument beschreibt reproduzierbare Ingame-Szenarien fuer die aktuelle XLAPI-Bridge-Policy. Ziel ist nicht Automatisierung auf JUnit-Niveau, sondern ein klarer manueller Testpfad, mit dem sich read_only- und read_write-Verhalten im Spiel jederzeit nachstellen laesst.

Aktuelle Policy

- bridged redstone_io ist read_write
- bridged screen ist read_only
- bridged light_sensor ist read_only
- bridged rain_sensor ist read_only
- bridged clock ist read_only
- bridged material_io ist read_only
- bridged crafting_io ist read_only
- bridged crafting_cpu ist read_only
- Endpoint-Umbenennung bleibt lokal
- XLAPI-Uplink-Gruppe und Relay-Status bleiben lokal

Gemeinsamer Testaufbau

1. Segment A aufbauen: Computer A, XLAPI A, Network Cable dazwischen.
2. Segment B aufbauen: Computer B, XLAPI B, mindestens ein Testgeraet, Network Cable dazwischen.
3. XLAPI A und XLAPI B auf dieselbe Uplink-Gruppe stellen.
4. Relay auf beiden XLAPI-Blocks aktivieren.
5. Sicherstellen, dass beide Segmente geladen sind.
6. Discovery auf Computer A und Computer B ausfuehren.

Erwartung:

- Computer A sieht lokale Endpunkte normal.
- Computer A sieht bridged Endpunkte aus Segment B mit scope bridged.
- Computer A sieht keine remote importierten XLAPI-Blocks als fernsteuerbare Bridge-Konfiguration.

Szenario 1: Read-Only Sensoren sichtbar

Aufbau:

- In Segment B einen Light Sensor und optional einen Clock-Block anschliessen.

Script auf Computer A:

```
def main():  
    rows = []  
    for name in list_devices():
```

```

        device = get_device(name)
        if device.is_remote():
            rows.append([
                device.api_name(),
                device.type(),
                device.remote_policy(),
                "yes" if device.remote_writable() else "no",
            ])

show_table("Remote Devices", ["API", "Type", "Policy", "Writable"], rows)

for name in list_devices():
    device = get_device(name)
    if not device.is_remote():
        continue
    if device.type() == "light_sensor":
        print("remote light:", device.light_level())
    if device.type() == "clock":
        print("remote day_time:", device.day_time())

main()

```

Erwartung:

- Die Geraete erscheinen als bridged.
- remote_policy ist read_only.
- remote_writable ist false.
- Lesezugriffe wie light_level() oder day_time() funktionieren.

Szenario 2: Read-Write Redstone ueber Bridge

Aufbau:

- In Segment B einen redstone_io an eine Lampe oder Redstone-Leitung haengen.
- Der Redstone-I/O-Block sollte einen stabilen Endpoint-Namen haben, zum Beispiel redstone_io.

Script auf Computer A:

```

def main():
    target = None
    for name in list_devices():
        device = get_device(name)
        if device.is_remote() and device.type() == "redstone_io":
            target = device
            break

    if target is None:
        print("no bridged redstone_io found")
        return

```

```

print(target.api_name(), target.remote_policy(), target.remote_writable())
target.set_mode("output")
target.write("north", 15)
print("north level:", target.read("north"))

main()

```

Erwartung:

- remote_policy ist read_write.
- remote_writable ist true.
- set_mode() und write() funktionieren ueber XLAPI.
- Die Lampe oder Leitung in Segment B reagiert sichtbar.

Szenario 3: Verbotene Material-I/O-Mutation

Aufbau:

- In Segment B einen material_io mit Inventar an Nord- und Suedseite anschliessen.

Script auf Computer A:

```

def main():
    target = None
    for name in list_devices():
        device = get_device(name)
        if device.is_remote() and device.type() == "material_io":
            target = device
            break

    if target is None:
        print("no bridged material_io found")
        return

    print(target.api_name(), target.remote_policy(), target.remote_writable())
    print("north inventory:", target.inventory("north"))

    try:
        moved = target.transfer_item("north", "south", 0, 1)
        print("unexpected transfer result:", moved)
    except Exception as exc:
        print("expected failure:", str(exc))

main()

```

Erwartung:

- inventory() funktioniert.
- transfer_item() scheitert mit einer klaren Meldung, dass der bridged Endpunkt read_only ist.

Szenario 4: Verbotene XLAPI-Rekonfiguration

Aufbau:

- Computer A muss einen lokalen XLAPI-Block haben.
- Segment B liefert zusätzlich einen bridged redstone_io oder Sensor, damit remote Geraete sichtbar sind.

Script auf Computer A:

```
def main():
    bridge = get_device("xlapi_block")
    print("local bridge group:", bridge.uplink_group())

    remote_bridge_like = [get_device(name) for name in list_devices() if
get_device(name).is_remote() and get_device(name).type() == "xlapi_block"]
    print("remote xlapi devices visible:", len(remote_bridge_like))

    try:
        bridge.set_uplink_group((bridge.uplink_group() + 1) % 16)
        print("local bridge reconfigured successfully")
    except Exception as exc:
        print("unexpected local failure:", str(exc))

main()
```

Erwartung:

- Lokale XLAPI-Konfiguration funktioniert weiterhin.
- Remote importierte XLAPI-Blocks tauchen nicht als fernkonfigurierbare Device-Konfiguration auf.

Szenario 5: Verbotene Crafting-Mutation

Aufbau:

- In Segment B ein crafting_io mit verlinktem crafting_cpu und material_io anschliessen.

Script auf Computer A:

```
def main():
    target = None
    for name in list_devices():
        device = get_device(name)
        if device.is_remote() and device.type() == "crafting_io":
            target = device
            break

    if target is None:
        print("no bridged crafting_io found")
```

```

        return

    print(target.api_name(), target.remote_policy(), target.remote_writable())
    print("plan:", target.plan())

    try:
        target.set_grid_slot(0, "minecraft:oak_planks", 1)
        print("unexpected crafting mutation succeeded")
    except Exception as exc:
        print("expected failure:", str(exc))

main()

```

Erwartung:

- plan() und andere lesende Abfragen funktionieren.
- set_grid_slot() oder andere Mutationen schlagen mit klarer read_only-Meldung fehl.

Szenario 6: Strukturierte Device-Liste eines Remote-Computers

Aufbau:

- Segment B enthaelt mindestens einen Computer, einen XLAPI-Block und zwei unterschiedliche Endpunkte, zum Beispiel redstone_io und light_sensor.

Script auf Computer A:

```

def main():
    bridge = get_device("xlapi_block")
    result = bridge.request_devices()
    print("request result:", result)

    for response in bridge.poll_responses():
        if response.get("command") != "devices":
            continue
        print("devices response:", response)

main()

```

Erwartung:

- request_devices() liefert eine request_id und delivered > 0.
- Die Response enthaelt devices als Liste.
- Jeder Device-Eintrag enthaelt mindestens api_name, type, scope, bridge_name, bridge_group, remote_policy und remote_writable.
- Die Device-Liste ist kompakt genug, um nicht abgeschnitten oder ungultig zu sein.

Szenario 7: Strukturierte Runtime-Details eines Remote-Computers

Aufbau:

- Computer B in Segment B fuehrt mindestens einmal ein Script mit Ausgabe oder Plan-Schritten aus.

Script auf Computer A:

```
def main():
    bridge = get_device("xlapi_block")
    result = bridge.request_runtime(output_limit=4, plan_limit=2)
    print("request result:", result)

    for response in bridge.poll_responses():
        if response.get("command") != "runtime":
            continue
        print("runtime response:", response)

main()
```

Erwartung:

- request_runtime() liefert eine request_id und delivered > 0.
- Die Response enthaelt runtime_status, summary, output_count und plan_step_count.
- output_lines respektiert output_limit.
- plan_steps respektiert plan_limit.
- output_truncated und plan_truncated spiegeln korrekt wider, ob mehr Daten vorhanden waeren.

Abnahmekriterien

- bridged Sensoren und Anzeigen sind lesbar, aber nicht fernschreibbar
- bridged redstone_io bleibt fernschreibbar
- bridged material_io und crafting_* bleiben lesbar, aber nicht mutierbar
- lokale XLAPI-Konfiguration bleibt lokal und wird nicht ueber die Bridge importiert
- strukturierte devices- und runtime-Responses bleiben gueltig, kompakt und reproduzierbar abrufbar
- Fehlermeldungen bei verbotenen Mutationen sind fuer Spieler und Entwickler klar genug, um die Policy sofort zu erkennen